
Every process p executes the following:

procedure *propose*(v_p)
 $V_p \leftarrow \langle \perp, \perp, \dots, \perp \rangle$ $\{p\text{'s estimate of the proposed values}\}$
 $V_p[p] \leftarrow v_p$
 $\Delta_p \leftarrow V_p$

Phase 1: $\{\text{asynchronous rounds } r_p, 1 \leq r_p \leq n - 1\}$
for $r_p \leftarrow 1$ to $n - 1$
 send (r_p, Δ_p, p) to all
 wait until $[\forall q : \text{received } (r_p, \Delta_q, q) \text{ or } q \in \mathcal{D}_p]$ $\{\text{query the failure detector}\}$
 $msgs_p[r_p] \leftarrow \{(r_p, \Delta_q, q) \mid \text{received } (r_p, \Delta_q, q)\}$
 $\Delta_p \leftarrow \langle \perp, \perp, \dots, \perp \rangle$
 for $k \leftarrow 1$ to n
 if $V_p[k] = \perp$ **and** $\exists (r_p, \Delta_q, q) \in msgs_p[r_p]$ with $\Delta_q[k] \neq \perp$ **then**
 $V_p[k] \leftarrow \Delta_q[k]$
 $\Delta_p[k] \leftarrow \Delta_q[k]$

Phase 2: send V_p to all
 wait until $[\forall q : \text{received } V_q \text{ or } q \in \mathcal{D}_p]$ $\{\text{query the failure detector}\}$
 $lastmsgs_p \leftarrow \{V_q \mid \text{received } V_q\}$
 for $k \leftarrow 1$ to n
 if $\exists V_q \in lastmsgs_p$ with $V_q[k] = \perp$ **then** $V_p[k] \leftarrow \perp$

Phase 3: *decide*(first non- $\perp$ component of V_p)

Figure 5: Solving Consensus using any $\mathcal{D} \in \mathcal{S}$.

Every process p executes the following:

procedure *propose*(v_p)

$estimate_p \leftarrow v_p$ $\{estimate_p \text{ is } p\text{'s estimate of the decision value}\}$

$state_p \leftarrow undecided$

$r_p \leftarrow 0$ $\{r_p \text{ is } p\text{'s current round number}\}$

$ts_p \leftarrow 0$ $\{ts_p \text{ is the last round in which } p \text{ updated } estimate_p, \text{ initially } 0\}$

$\{Rotate \text{ through coordinators until decision is reached}\}$

while $state_p = undecided$

$r_p \leftarrow r_p + 1$

$c_p \leftarrow (r_p \bmod n) + 1$ $\{c_p \text{ is the current coordinator}\}$

Phase 1: $\{All \text{ processes } p \text{ send } estimate_p \text{ to the current coordinator}\}$

send $(p, r_p, estimate_p, ts_p)$ to c_p

Phase 2: $\{The \text{ current coordinator gathers } \lceil \frac{(n+1)}{2} \rceil \text{ estimates and proposes a new estimate}\}$

if $p = c_p$ **then**

wait until [for $\lceil \frac{(n+1)}{2} \rceil$ processes q : received $(q, r_p, estimate_q, ts_q)$ from q]

$msgs_p[r_p] \leftarrow \{(q, r_p, estimate_q, ts_q) \mid p \text{ received } (q, r_p, estimate_q, ts_q) \text{ from } q\}$

$t \leftarrow \text{largest } ts_q \text{ such that } (q, r_p, estimate_q, ts_q) \in msgs_p[r_p]$

$estimate_p \leftarrow \text{select one } estimate_q \text{ such that } (q, r_p, estimate_q, t) \in msgs_p[r_p]$

send $(p, r_p, estimate_p)$ to all

Phase 3: $\{All \text{ processes wait for the new estimate proposed by the current coordinator}\}$

wait until [received $(c_p, r_p, estimate_{c_p})$ from c_p **or** $c_p \in \mathcal{D}_p$] $\{Query \text{ the failure detector}\}$

if [received $(c_p, r_p, estimate_{c_p})$ from c_p] **then** $\{p \text{ received } estimate_{c_p} \text{ from } c_p\}$

$estimate_p \leftarrow estimate_{c_p}$

$ts_p \leftarrow r_p$

send (p, r_p, ack) to c_p

else send $(p, r_p, nack)$ to c_p $\{p \text{ suspects that } c_p \text{ crashed}\}$

Phase 4: $\left\{ \begin{array}{l} The \text{ current coordinator waits for } \lceil \frac{(n+1)}{2} \rceil \text{ replies. If they indicate that } \lceil \frac{(n+1)}{2} \rceil \\ \text{processes adopted its estimate, the coordinator } R\text{-broadcasts a decide message} \end{array} \right\}$

if $p = c_p$ **then**

wait until [for $\lceil \frac{(n+1)}{2} \rceil$ processes q : received (q, r_p, ack) **or** $(q, r_p, nack)$]

if [for $\lceil \frac{(n+1)}{2} \rceil$ processes q : received (q, r_p, ack)] **then**

$R\text{-broadcast}(p, r_p, estimate_p, decide)$

$\{If \text{ } p \text{ } R\text{-delivers a decide message, } p \text{ decides accordingly}\}$

when $R\text{-deliver}(q, r_q, estimate_q, decide)$

if $state_p = undecided$ **then**

$decide(estimate_q)$

$state_p \leftarrow decided$

Figure 6: Solving Consensus using any $\mathcal{D} \in \Diamond\mathcal{S}$.